

Tema 8: Algoritmdesigntechniker

Wilhelm Durelius widu7139

3 mars 2026

1 Backtracking

Backtracking är en algoritmdesigntechnik där man gör val nedåt, och om man når ett val som inte uppfyller de förbestämda kraven går man uppåt, eller bakåt snarare, i valkedjan och testar syskonvalen.

Tekniken kan ses som ett träd där varje nod representerar ett val. Traversering sker på djupet i trädet och när en nod som inte uppfyller kraven nås, utförs ett bakåttsteg till föräldernoden och testar nästa barn. Backtracking kan returnera tomt, om alla möjliga vägar nedåt inte uppfyller kraven. Tidskomplexiteten för backtracking är i värsta fall exponentiell, men i praktiken kan den vara betydligt bättre tack vare *pruning*; att man tidigt utesluter grenar i trädet som omöjligt kan leda till en giltig lösning. Ju effektivare pruning, desto snabbare algoritm. Pruning eller beskärning på svenska innebär alltså att man skär bort delar av trädet man bedömer inte uppfyller kraven. Backtracking

lämpar sig väl för problem där datamängden är stor och strukturerad, där ogiltiga dellösningar kan identifieras tidigt. Backtracking implementeras oftast rekursivt, då stacken automatiskt håller koll på var man är någonstans i traverseringen, iterära implementationer är mycket svårare att få till, och är inte att rekommendera.

2 Sluppmässiga algoritmer

Sluppmässiga algoritmer är en algoritmdesignsteknik där man använder slump i algoritmen för att inte behöva räkna ut optimala värden på förhand, vilket sparar beräkningstid. Situationer där sluppmässiga algoritmer skiner är när man behöver göra val baserat på datat, men vinsten av att räkna ut de optimala valen är mindre än att använda sluppmässiga tal, ofta blandat med heuristik.

Deterministiska val i vissa specifika situationer kan leda till att man alltid får ett worst-case scenario i tidskomplexitet, medan slumpen istället innebär att man får worst-case scenario vid få tillfällen. Att alltid få dålig körtid är såklart sämre än att ibland få en dålig körtid. Genom att istället använda slump bryter man dessa mönster, vilket i genomsnitt leder till bättre resultat. Sluppmässiga algoritmer lämpar sig väl för problem där deterministiska algoritmer är alltför långsamma, eller där en approximativ lösning är tillräcklig.

Sluppmässiga algoritmer kan alltså ses som ett skydd mot dålig indata i vissa fall, och en effektivisering i och med att man inte behöver räkna ut de optimala värdena i andra fall.

3 Divide and conquer

Divide and conquer är en algoritmdesignsteknik som går ut på att dela upp ett problem i mindre delar, lösa varje del för sig, och sedan sätta ihop delresultaten till ett slutgiltigt svar. För att tekniken ska fungera krävs att alla delproblem är av samma typ som huvudproblemet, det vill säga att samma algoritm kan användas för alla delproblem. Detta gör att divide and conquer ofta går hand i hand med rekursion, och vissa definitioner har det som ett grundkrav.

Det krävs också att delresultaten inte överlappar varandra i någon större utsträckning. Om samma beräkningar görs om flera gånger blir algoritmen onödigt långsam, och då kan en annan teknik vara mer lämplig.

Processen sker i tre steg: först delas problemet upp, sedan löses varje del, och till sist kombineras resultaten. Uppdelningen sker vanligtvis rekursivt tills delarna är tillräckligt små för att lösas direkt. Det är viktigt att ihopslagningen av resultaten inte är alltför kostsam, annars riskerar det sista steget att dra ner algoritmens prestanda.

Divide and conquer lämpar sig väl när ett problem enkelt kan delas upp i oberoende delproblem, och när det är smidigt att kombinera deras resultat. En ytterligare fördel är att de oberoende delarna ofta kan beräknas parallellt, vilket kan ge ytterligare prestandavinster.